

Student 21 **Lakshmi Narayanan E** [192524249RD](#)

3 / 3 submitted

Q1. Multiple Threads Using a Single Class

Score: 10.00 TC: 3/3 passed

CODE

```
import java.util.*;
public class Main {
    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        MyThread t1 = new MyThread("Prime", n);
        MyThread t2 = new MyThread("Armstrong", n);
        MyThread t3 = new MyThread("Reverse", n);
        t1.start();
        t1.join();
        t2.start();
        t2.join();
        t3.start();
        t3.join();
    }
}

class MyThread extends Thread {
    int n;
    MyThread(String name, int n) {
        super(name);
        this.n = n;
    }

    public void run() {
        String name = getName();
        if (name.equals("Prime")) {
            if (isPrime(n))
                System.out.println("Prime Thread: " + n + " is a prime number");
            else
                System.out.println("Prime Thread: " + n + " is not a prime number");
        } else if (name.equals("Armstrong")) {
            if (isArmstrong(n))
                System.out.println("Armstrong Thread: " + n + " is an Armstrong number");
            else
                System.out.println("Armstrong Thread: " + n + " is not an Armstrong number");
        } else if (name.equals("Reverse")) {
            System.out.println("Reverse Thread: Reverse of " + n + " = " + reverse(n));
        }
    }

    boolean isPrime(int num) {
        if (num <= 1)
            return false;
        for (int i = 2; i <= Math.sqrt(num); i++) {
            if (num % i == 0)
                return false;
        }
        return true;
    }

    boolean isArmstrong(int num) {
        int temp = num;
        int sum = 0;
        int digits = String.valueOf(num).length();

        while (temp > 0) {
            int rem = temp % 10;
            sum += Math.pow(rem, digits);
            temp /= 10;
        }
        return sum == num;
    }
}
```

```
int reverse(int num) {  
    int rev = 0;  
    int temp = num;  
    while (temp > 0) {  
        rev = rev * 10 + temp % 10;  
        temp /= 10;  
    }  
    return rev;  
}  
}
```

INPUT

153

OUTPUT

Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 = 351

Q2. Multiple thread classes

Score: 20.00 TC: 3/3 passed

CODE

```
import java.util.*;
public class Main {
    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        PrimeThread t1 = new PrimeThread(n);
        FactorialThread t2 = new FactorialThread(n);
        ReverseThread t3 = new ReverseThread(n);
        t1.start();
        t1.join();
        t2.start();
        t2.join();
        t3.start();
        t3.join();
    }
}

class PrimeThread extends Thread {
    int n;
    PrimeThread(int n) {
        this.n = n;
    }
    public void run() {
        boolean prime = true;
        if (n <= 1)
            prime = false;
        else {
            for (int i = 2; i <= Math.sqrt(n); i++) {
                if (n % i == 0) {
                    prime = false;
                    break;
                }
            }
        }
        if (prime)
            System.out.println("Prime: " + n + " is a prime number");
        else
            System.out.println("Prime: " + n + " is not a prime number");
    }
}

class FactorialThread extends Thread {
    int n;
    FactorialThread(int n) {
        this.n = n;
    }
    public void run() {
        long fact = 1;
        for (int i = 1; i <= n; i++) {
            fact *= i;
        }
        System.out.println("Factorial: " + n + "! = " + fact);
    }
}

class ReverseThread extends Thread {
    int n;
    ReverseThread(int n) {
        this.n = n;
    }
    public void run() {
        int temp = n;
        int rev = 0;
    }
}
```

```
        while (temp > 0) {
            rev = rev * 10 + temp % 10;
            temp /= 10;
        }
        System.out.println("Reverse: Reverse of " + n + " = " + rev);
    }
}
```

INPUT

7

OUTPUT

Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7

Q3. Thread Synchronization

Score: 10.00 TC: 3/3 passed

CODE

```
public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        ThreadA t1 = new ThreadA(counter);
        ThreadB t2 = new ThreadB(counter);
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("Final Count = " + counter.getCount());
    }
}

class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
    public int getCount() {
        return count;
    }
}

class ThreadA extends Thread {
    Counter counter;
    ThreadA(Counter counter) {
        this.counter = counter;
    }
    public void run() {
        for (int i = 0; i < 1000; i++) {
            counter.increment();
        }
        System.out.println("ThreadA completed");
    }
}

class ThreadB extends Thread {
    Counter counter;
    ThreadB(Counter counter) {
        this.counter = counter;
    }
    public void run() {
        for (int i = 0; i < 1000; i++) {
            counter.increment();
        }
        System.out.println("ThreadB completed");
    }
}
```

OUTPUT

```
ThreadA completed
ThreadB completed
Final Count = 2000
```